

Лабораторная 1. Как измерять сложность алгоритмов

Что изучаем

Одна и та же задача может решаться за доли секунды или за часы. Причина часто в том, как число действий растёт вместе с размером входа. В этой работе четыре собственные функции дают три разных закона роста: линейный, квадратичный и логарифмический. Требования работы и способ задания варианта взяты из КИМ-01 курса МТУСИ.[1]

Размер n — количество элементов массива. Для возведения в степень n означает показатель степени, а не размер массива. Это различие нужно всегда проговаривать: оценка без определения размера входа неоднозначна.

Сначала считаем действия

Представьте стопку из n карточек. Чтобы сложить написанные числа, нужно прочитать каждую карточку один раз. После k шагов переменная `total` равна сумме первых k элементов. Это **инвариант** — утверждение, которое верно до цикла и сохраняется после каждой итерации. Когда k достигает n , он доказывает правильность ответа.

Поиск максимума похож. Начинаем с первого элемента, затем сравниваем очередной элемент с текущим максимумом. После обработки первых k значений `largest` хранит их максимум. Сравнений ровно $n-1$. Начинать с нуля нельзя: на массиве $[-8, -3]$ функция ошибочно вернула бы 0. Пустой массив не имеет максимума; реализация выбрасывает `ValueError`.

В подсчёте равных пар интересуют разные индексы $i < j$. Для $i=0$ проверяется $n-1$ партнёр, для $i=1$ — $n-2$, и так далее. Всего $(n-1)+(n-2)+\dots+1 = n(n-1)/2$ сравнений. Ограничение $i < j$ исключает сравнение элемента с собой и двойной подсчёт пары.

Например, у $[4, 4, 7, 4]$ три равные пары: $(0,1)$, $(0,3)$, $(1,3)$. Три одинаковых значения образуют $3 \cdot 2/2 = 3$ пары. Счётчик увеличивается только при равенстве, но само сравнение выполняется для каждой пары: даже массив без повторов требует квадратичного времени.

Что означают O , Θ и Ω

$O(g(n))$ — верхняя граница с точностью до постоянного множителя при достаточно больших n . Формально существуют $C > 0$ и n_0 , такие что $T(n) \leq C \cdot g(n)$ при $n \geq n_0$. Ω — аналогичная нижняя граница. Θ означает, что обе границы совпали по порядку.

Например, $3n+10$ принадлежит $\Theta(n)$, $O(n^2)$ и $\Omega(1)$. Поэтому утверждение «алгоритм $O(n^2)$ » само по себе не доказывает квадратичный рост: линейный алгоритм тоже удовлетворяет такой верхней оценке. Здесь для суммы и максимума используем точную по порядку оценку $\Theta(n)$, для двойного цикла — $\Theta(n^2)$.

Худший случай — максимум времени среди всех входов размера n ; лучший — минимум. Средний — математическое ожидание при явно заданном распределении входов. Нельзя считать среднее без объяснения, какие входы и с какими вероятностями рассматриваются. Аналитическая модель и эксперимент дополняют друг друга.[2]

Бинарная степень по шагам

Пусть нужно вычислить 3^{13} . Число 13 записывается как $8+4+1$, поэтому $3^{13} = 3^8 \cdot 3^4 \cdot 3$. На каждом шаге смотрим на младший бит показателя. Если он равен 1, домножаем результат на текущую степень основания. Затем показатель делим на 2, а основание возводим в квадрат.

Остаток показателя	Основание	Результат перед шагом	Действие
13	3	1	Домножить на 3
6	9	3	Не домножать
3	81	3	Домножить на 81
1	6561	243	Домножить на 6561

Итог — 1 594 323. Инвариант: $\text{result} \cdot \text{base}^{\text{exponent}}$ равен исходной степени; при модуле равенство понимается по модулю. После обнуления exponent остаётся только result . Итераций $\lceil \log_2 n \rceil + 1$ для $n > 0$; случай $n = 0$ возвращает 1, а при $\text{mod} = 1$ — 0.

В модели операций над числами ограниченной длины время $O(\log n)$, дополнительная память $O(1)$. В Python целые могут расти: число 3^n содержит $\Theta(n)$ бит. Его умножение уже не $O(1)$. Поэтому в эксперименте используем модуль 1 000 000 007 и удерживаем промежуточные значения ограниченными. При переменном размере модуля нужно учитывать и его битовую длину.

Как читать код и запускать работу

Откройте `algorithms.py`: `array_sum` и `array_max` поддерживают накопленный ответ; `count_equal_pairs` перебирает верхний треугольник пар индексов; `binary_row` уменьшает exponent на каждом шаге. Затем из корня репозитория выполните:

```
python -m pytest labs/lab01 -q
python -m labs.lab01.experiment --variant 15
```

Перед экспериментом данные создаются общими командами из README репозитория. Seed равен $30+15=45$. Сумма и максимум измеряются на пяти размерах 1 000...100 000; пары — на 500, 1 000, 3 000, 10 000, 30 000. Последний ряд соответствует ограничению стартовой заготовки: 100 000 элементов дали бы почти пять миллиардов сравнений за один запуск.

Почему нельзя просто один раз засечь время

Таймер учитывает работу интерпретатора, состояние кэшей и задержки планировщика. Поэтому вход загружается заранее, один запуск служит прогревом, затем сохраняются пять длительностей и их медиана. Для очень короткой бинарной степени один замер объединяет 20 000 вызовов, а время делится на их число.[3]

Если $t \approx C \cdot n^p$, то $\log t \approx \log C + p \cdot \log n$. На $\log\text{-}\log$ графике наклон примерно p : около 1 для линейного алгоритма, около 2 для квадратичного. Это приближение, а не математическое доказательство. Для $t \approx C \cdot \log_2 n$ строится обычный график t от $\log_2 n$: там зависимость почти линейна. На $\log\text{-}\log$ графике логарифм выглядит медленно изгибающейся кривой, и его наклон не даёт постоянной степени.

Верификация

Проверяются пустые входы, один элемент, отрицательные числа, все равные и все разные элементы. На 400 случайных наборах сумма и максимум сравниваются с sum/max , степень — с row . Для пар независимый эталон использует частоты: каждая группа из s равных значений даёт $s(s-1)/2$. Это полезнее, чем сравнивать два одинаковых двойных цикла.

Вопросы на защите и ответы

- 1. Чем O отличается от Θ ?** O ограничивает рост сверху; Θ фиксирует порядок одновременно сверху и снизу. Для цикла с точно n итерациями содержательнее сказать $\Theta(n)$.
- 2. В чём разница между худшим и средним случаем?** Худший выбирает самый дорогой вход фиксированного размера. Средний усредняет по распределению входов. Для этих трёх проходов число проверок почти не зависит от значений, но количество присваиваний может отличаться.
- 3. Как наклон графика связан со сложностью?** После логарифмирования $t = Cn^p$ показатель p становится коэффициентом при $\log n$. На конечных размерах на оценку влияют постоянные затраты и шум.
- 4. Зачем прогрев и медиана?** Прогрев уменьшает влияние первого запуска, медиана менее чувствительна к единичной задержке. Они не устраняют систематическую нагрузку или изменение частоты CPU; условия измерения всё равно нужно указать.
- 5. Можно ли посчитать пары быстрее?** Да: накопить частоты в хеш-таблице, затем сложить $s(s-1)/2$. Это ожидаемое $O(n)$, но в данной лабораторной нужен именно двойной цикл для исследования $\Theta(n^2)$.
- 6. Почему максимум нельзя начать с нуля?** Ноль может не принадлежать массиву и превосходить все его отрицательные элементы. Начальное значение берём из самого входа.
- 7. Почему степень по модулю даёт тот же остаток?** Замена множителя его остатком не меняет остаток произведения: $(ab) \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$.
- 8. Что показать преподавателю?** Указать seed 45, запустить тесты, объяснить один инвариант, открыть графики и CSV, показать отличие линейного и квадратичного роста. Числа из конкретного запуска приведены в приложенном отчёте.

Источники

- [1] МТУСИ, кафедра программной инженерии. КИМ-01, 2026.
- [2] Robert Sedgewick, Kevin Wayne. Analysis of Algorithms, материалы авторов Algorithms, 4th edition.
- [3] МТУСИ. Настройка окружения и воспроизводимость, 2026.

Отчёт по ЛР 1. Анализ сложности

Цель и выполненная работа

Реализованы алгоритмы и проверки, описанные в конспекте. Эксперимент выполнен на данных варианта 15, seed 45. В таблицах приведены фактические результаты запуска; время указано в секундах, если не оговорено иное.

Условия и методика

CPU: 12th Gen Intel(R) Core(TM) i7-12650H. ОС: Windows-11-10.0.26200-SP0. Python 3.12.12, NumPy 2.5.3, Matplotlib 3.11.1. Дата и время UTC: 2026-09-10T18:24:34.061247+00:00.

Один прогрев и пять повторов на точку, итог — медиана `time.perf_counter`. Ввод, генерация и подготовка свежего состояния исключены из таймера. Фоновые процессы и питание не контролировались; задержки рабочего компьютера могут влиять на отдельные точки. Контрольные суммы данных находятся в `environment.json`.

Результаты и их объяснение

Алгоритм	Наклон log-log
sum	0.876541
max	0.930777
equal_pairs	1.99447

Для пар наклон 1,994 близок к теоретическому 2. Для суммы 0,877 и максимума 0,931 близки к линейному порядку, но заметно отклоняются от 1: на небольшом диапазоне влияют постоянные затраты, шум и условия выполнения. Это наблюдение не меняет аналитический подсчёт $\Theta(n)$.

Алгоритм	n	Медиана, с
sum	1000	8.9e-05
sum	3000	0.0002551
sum	10000	0.0008332
sum	30000	0.0021419
sum	100000	0.0047837
max	1000	4.35e-05
max	3000	0.0002952
max	10000	0.0004269
max	30000	0.0013388
max	100000	0.0043868
equal_pairs	500	0.0061865
equal_pairs	1000	0.0263289
equal_pairs	3000	0.227767

Алгоритм	n	Медиана, с
equal_pairs	10000	2.47259
equal_pairs	30000	22.4571

Показатель	$\log_2 n$	с/вызов
1000	9.96578	1.68027e-06
10000	13.2877	2.17713e-06
100000	16.6096	2.68383e-06
1000000	19.9316	3.2013e-06
10000000	23.2535	3.82185e-06

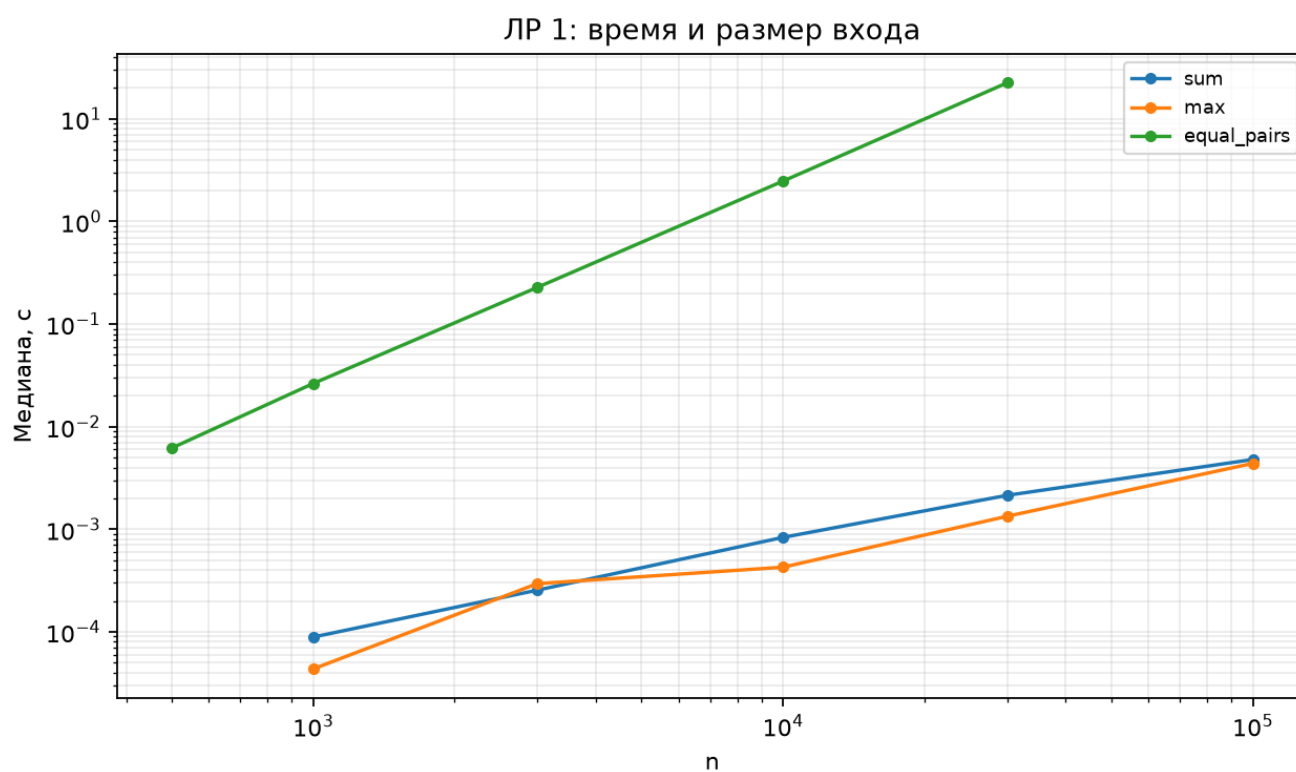
Бинарная степень измерена пакетами по 20 000 вызовов по модулю 1 000 000 007. Число итераций определяется битовой длиной показателя; число домножений результата зависит также от единичных битов, поэтому точки не обязаны лежать точно на прямой.

Проверка и вывод о применимости

Проверки находятся в test_lab.py. Запуск: `python -m pytest labs/lab01 -q`. Применены граничные случаи, инварианты и независимые эталоны; состав тестов подробно разобран в конспекте. Проверки всего комплекта также сохранены в verification.txt в корне репозитория.

Вывод: принять для учебного использования в рамках явно описанных контрактов. Измерения подтверждают основные тенденции роста; ограничения реализаций и сравнения указаны выше и в конспекте. Автоматическая проверка не оценивает готовность к устной защите.

Графики



Исходные данные отчёта

CSV содержит медиану и все пять длительностей trial_1...trial_5. Вспомогательные таблицы без времени содержат точные счётчики или свойства структуры.

- measurements.csv

- powers.csv
- slopes.csv
- Паспорт окружения и контрольные суммы